

Relazione finale sul progetto del corso di Simulazione di Sistemi

Presentata da:

Beatrice Ravaglia
0001217838

Laurea Magistrale in Informatica
Anno Accademico 2025/2026

Indice

1	Introduzione: motivazioni e contesto	1
1.1	Caratteristiche dei Sistemi Reali	1
2	Descrizione formale del modello	2
2.1	Modello generale	2
2.2	Arrival process	2
2.3	Dimensioni dei batch	2
2.4	Service Rule	2
2.5	Service Process	3
2.6	Single Vacation Rule (SVR)	4
2.7	Set-Up Process	4
2.8	Parametri del modello	5
3	Implementazione	5
3.1	Descrizione della rete e parametri di simulazione	5
3.2	Componente Source	6
3.3	Componente Queue	7
3.4	Componente Server	11
3.5	Interazione tra Queue e Server	14
3.6	Componente Sink	15
4	Stima delle misure di prestazione	16
4.1	Dati raccolti durante le simulazioni	16
4.2	Analisi del transiente iniziale	17
4.2.1	Il problema del transiente iniziale	17
4.2.2	Individuazione del punto di warm-up	17
4.3	Calcolo delle stime puntuali	21
4.4	Calcolo degli intervalli di confidenza	22
5	Risultati	23
5.1	Transiente iniziale	23
5.2	Misure di prestazione	24
5.2.1	Stime per $m = 1.0$	24
5.2.2	Stime per $m = 1.4$	25
5.2.3	Stime per $m = 2.0$	25
5.2.4	Stime per $m = 2.5$	25
5.3	Osservazioni sui risultati e conclusioni	26
5.3.1	Osservazioni	26
5.3.2	Controllo della coerenza dei risultati	26

1 Introduzione: motivazioni e contesto

Questa sezione descrive le principali caratteristiche dei sistemi reali a cui si riferisce il modello simulato nel progetto.

1.1 Caratteristiche dei Sistemi Reali

In molti sistemi reali, le entità non vengono processate una alla volta, ma in gruppi, e il tempo di servizio spesso dipende dalla dimensione del gruppo. Per questo motivo, il modello “one-customer-at-a-time” risulta spesso irrealistico.

Diversi esempi concreti illustrano questa situazione:

- **Traffico stradale:** le automobili attraversano gli incroci in “ondate” di flusso.
- **Industria manifatturiera:** una macchina può lavorare solo quando ha accumulato un numero sufficiente di pezzi e può fermarsi (vacanza) in assenza di lavoro.
- **Wireless Sensor Networks (WSN):** i nodi trasmettono dati accumulati in blocchi per ottimizzare risorse e energia.

Emblematico è l'esempio delle **Wireless Sensor Networks (WSN)**, che rendono questo tipo di modello particolarmente attuale. Le WSN rappresentano una tipologia di rete informatica caratterizzata da un'architettura distribuita, costituita da un insieme di sensori disposti in prossimità o all'interno del fenomeno da osservare. I sensori rilevano dati dall'ambiente circostante e comunicano tra loro mediante tecnologia wireless a corto raggio. La comunicazione è generalmente asimmetrica: i nodi comuni trasmettono i dati raccolti a uno o più nodi speciali, detti *sink* o *gateway*, che hanno il compito di raccogliere le informazioni e inoltrarle ad un server o a un calcolatore centrale per successive elaborazioni.

Le WSN trovano applicazione in numerosi contesti, tra cui:

- monitoraggio del traffico urbano;
- monitoraggio ambientale (inquinamento, temperatura, umidità, vento);
- sistemi militari;
- sanità (monitoraggio del corpo umano).

Per motivi di efficienza energetica, le WSN elaborano e trasmettono i dati in batch. Poiché ogni sensore dispone di una riserva di energia limitata e non rinnovabile, trasmettere continuamente dati comporterebbe uno spreco energetico. Pertanto, i

nodi accumulano i dati e li trasmettono in blocco, entrando in modalità di *sleep* quando possibile per ridurre il consumo energetico.

2 Descrizione formale del modello

Il modello analizzato, mediante simulazione, è una variante di quello descritto nell'articolo di riferimento [1].

2.1 Modello generale

Il sistema considerato prevede **due classi di utenti**, indicate con $i = 1, 2$, che attendono in un'unica coda servita da un singolo server. Gli utenti di ciascuna classe arrivano nel sistema in *batch*. Questo tipo di modello consente di rappresentare sistemi in cui le entità non vengono processate una alla volta, ma in gruppi, come nelle Wireless Sensor Networks.

2.2 Arrival process

Il processo di arrivo degli utenti di classe i è di tipo ***batch***. Il tempo di inter-arrivo tra i batch di ciascuna classe è descritto da una variabile casuale $A(t)$, distribuita secondo una legge esponenziale con media m :

$$A(t) \sim \text{Exp}(m)$$

2.3 Dimensioni dei batch

Ogni arrivo di un utente di classe i corrisponde a un batch di dimensione X_i , che è una variabile casuale con distribuzione di probabilità $B_i(X_i)$:

$$X_i \sim B_i(X_i)$$

2.4 Service Rule

Il sistema implementa un **meccanismo di servizio prioritario**, in cui gli utenti di classe 1 hanno priorità su quelli di classe 2. Per entrambe le classi si utilizza la stessa regola parametrica $[a_i, b_i]$ rule, definita da una coppia di parametri (a_i, b_i) per la classe i , $i = 1, 2$.

Il funzionamento del meccanismo è il seguente:

- Il server serve batch di utenti di classe 1 finché la coda contiene almeno a_1 utenti di classe 1. In ciascun servizio, il numero di utenti serviti è definito come:

$$r_1 = \min(b_1, \text{utenti di classe 1 in coda})$$

Tutti gli utenti nel batch selezionato vengono serviti simultaneamente.

- Quando non è più possibile servire utenti di classe 1, il servizio viene esteso agli utenti della classe 2, secondo la stessa logica:

$$r_2 = \min(b_2, \text{utenti di classe 2 in coda}),$$

a condizione che la coda contenga almeno a_2 utenti di classe 2.

- Qualora nessuna delle condizioni sia soddisfatta (cioè non è possibile servire batch di alcuna classe), il sistema entra in uno **stato di vacation**, come descritto nella relativa sezione.

2.5 Service Process

Il tempo di servizio di ciascun batch dipende dalla dimensione del batch stesso ed è modellato come la somma di due componenti: una parte deterministica e una parte casuale.

Per un batch di classe i di dimensione r_i con $a_i \leq r_i \leq b_i$, il tempo di servizio t_i è definito come:

$$t_i = s_i + q_i$$

dove:

- s_i è la componente deterministica:

$$s_i = \frac{r_i}{b_i} sp_i$$

con sp_i che rappresenta il tempo di servizio previsto per un batch completo di dimensione b_i della classe i .

- q_i è la componente casuale, distribuita uniformemente nell'intervallo $[l_i, u_i]$, con $0 < l_i < u_i$:

$$q_i \sim U(l_i, u_i)$$

2.6 Single Vacation Rule (SVR)

Il server entra in **vacanza** se non è possibile servire utenti di classe 1 né utenti di classe 2, ovvero quando le condizioni definite dalla *Service Rule* non possono essere soddisfatte.

Il tempo di vacation dipende dal numero di utenti di classe 1 presenti in coda. Sia k il numero di utenti di classe 1 in coda, con $0 \leq k \leq a_1 - 1$. Il tempo di vacation t è definito come somma di due componenti indipendenti:

$$t = t' + t''$$

dove:

- t' è la componente deterministica:

$$t' = \frac{k+1}{b_1} sv$$

con sv il tempo di vacation corrispondente a un batch completo di classe 1.

- t'' è la componente casuale, distribuita uniformemente nell'intervallo $[z, w]$, con $0 < z < w$:

$$t'' \sim U(z, w)$$

La distribuzione di probabilità del tempo di vacation viene indicata con $\Psi^{[k]}(t)$.

Intuitivamente, code brevi permettono al server di rimanere in vacation più a lungo, mentre code consistenti riducono il tempo di attesa.

2.7 Set-Up Process

Quando il server termina la vacanza, deve verificare se è stata raggiunta una **soglia** di N utenti di classe 1 in coda:

- Se la soglia N è stata raggiunta, il server inizia la fase di set-up.
- Se la soglia non è stata raggiunta, il server rimane in attesa finché il numero di utenti in coda raggiunge N , dopodiché inizia il set-up.

Il tempo di set-up t è una variabile casuale distribuita esponenzialmente con media h :

$$t \sim R(t) = \text{Exp}(h)$$

Al termine della fase di set-up, il server riprende immediatamente il servizio dei batch secondo la *Service Rule*.

2.8 Parametri del modello

I principali parametri utilizzati nel modello sono riassunti nella Tabella 1.

Tabella 1: Parametri del modello di simulazione

Parametro	Simbolo	Valore / Intervallo
Processo di arrivo	$A(t)$	Esponenziale, $m = \{1.0, 1.4, 2.0, 2.5\}$
Dimensione batch classe 1	$B_1(X_1)$	Uniforme discreta, $U[1, 4]$
Dimensione batch classe 2	$B_2(X_2)$	Uniforme discreta, $U[1, 3]$
Tempo di servizio classe 1	$S_{r1}(t)$	$a_1 = 7, b_1 = 16, sp_1 = 0.1, l_1 = 1.0, u_1 = 3.92$
Tempo di servizio classe 2	$S_{r2}(t)$	$a_2 = 5, b_2 = 14, sp_2 = 0.1, l_2 = 1.0, u_2 = 2.54$
Vacation	$\Psi^{[k]}(t)$	$N = 20, sv = 0.2, z = 2.0, w = 3.8$
Set-up process	$R(t)$	Esponenziale, $h = 1.0$

3 Implementazione

Il modello di simulazione è stato implementato tramite la piattaforma **OMNeT++**, un ambiente di simulazione a eventi discreti open-source. [2]

Basato su un'architettura modulare, OMNeT++ consente di costruire modelli attraverso la composizione gerarchica di moduli definiti tramite il linguaggio NED (Network Description). Il comportamento dinamico dei moduli è implementato in C++, mentre la configurazione dei parametri di simulazione avviene tramite file .ini.

3.1 Descrizione della rete e parametri di simulazione

La struttura topologica della rete simulata è stata definita in un file **.ned**, specificando i moduli che la compongono e le connessioni tra di essi. In particolare, il modello è costituito da due moduli **Source**, che generano le entità in ingresso, un modulo **Queue**, che gestisce l'accodamento delle richieste, un modulo **Server**, responsabile del servizio, e un modulo **Sink**, che rappresenta l'uscita del sistema.

La topologia della rete è rappresentata nella figura 1.

I parametri del modello riportati nella Tabella 1 sono stati invece opportunamente definiti nel file .ini di configurazione.

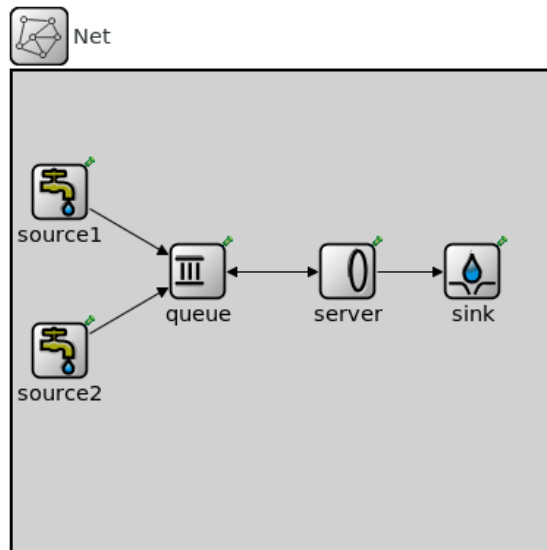


Figura 1: Struttura del modello, definita nel file .ned

3.2 Componente Source

I batch di utenti appartenenti alle due differenti classi (1, 2) vengono generati tramite due moduli **Source1** e **Source2**, istanze del componente **Source**.

Per modellare le entità in ingresso al sistema è stato definito il messaggio **Job.msg**, che estende il tipo **packet** includendo due campi informativi: **userClass**, che identifica la classe dell'utente (1 o 2), e **arrivalTime**, che memorizza il tempo di ingresso nel sistema per il calcolo delle metriche prestazionali.

Il comportamento dinamico della sorgente è implementato nel metodo **handleMessage()**. Quando si verifica l'evento di generazione (**generateEvent**), il modulo estrae la dimensione del batch secondo una distribuzione uniforme discreta nell'intervallo **[batchMin, batchMax]**. Vengono quindi creati tanti oggetti **Job** quanti indicati dal batch, assegnando a ciascuno la classe utente e lo stesso timestamp di arrivo (**simTime()**). I job vengono quindi inviati attraverso il gate **out** verso la coda.

Al termine della generazione del batch, la sorgente invia alla coda un messaggio di controllo (**SourceBatchComplete**), e pianifica il batch successivo dopo un intervallo di tempo esponenziale con media **meanInterArrival**, modellando così il processo di arrivi stocastico.

Listing 1: Funzione handleMessage() del modulo Source.

```
1 void Source::handleMessage(cMessage *msg)
2 {
3     if (msg == generateEvent)
4     {
5         int batchSize = intuniform(batchMin, batchMax);
6         for (int i = 0; i < batchSize; i++)
7         {
8             Job *job = new Job("job");
9             job->setUserClass(userClass);
10            job->setArrivalTime(simTime());
11
12            send(job, "out");
13        }
14        cMessage *batchDone = new cMessage("
15            SourceBatchComplete");
16        send(batchDone, "out");
17
18        simtime_t nextArr = exponential(meanInterArrival);
19        scheduleAt(simTime() + nextArr, generateEvent);
20    }
21 }
```

3.3 Componente Queue

Il componente Queue implementa la logica centrale del sistema. All'interno del modulo sono mantenute due code distinte, `queueClass1` e `queueClass2`, una per ciascuna classe di utenti. Anche in questo caso i batch sono “simulati”: i job vengono inviati uno alla volta al **Server**, e al termine dell'invio viene trasmesso un messaggio di controllo (**BatchComplete**) per segnalare la fine del lotto.

La gestione delle richieste del server avviene nel metodo `handleMessage()`. Quando il server invia un messaggio **ServerRequestBatch**, la coda verifica se è in corso la ricezione di un batch dalla sorgente. I flag booleani `batchInProgress` e `waitingForBatchComplete` permettono di gestire correttamente il caso in cui il server richieda un batch mentre la coda sta ancora ricevendo i job: poiché il batch è rappresentato come una sequenza di job singoli, è necessario attendere il messaggio **SourceBatchComplete** dalla Source prima di poter servire la richiesta.

Listing 2: Funzione handleMessage() del modulo Queue.

```
1 void Queue::handleMessage(cMessage *msg)
2 {
3     if (strcmp(msg->getName(), "ServerRequestBatch") == 0) {
4         delete msg;
5         if(!batchInProgress)
6             serveBatch();
7         else
8             waitingForBatchComplete = true;
9     }
10    else if (strcmp(msg->getName(), "JobDeparture") == 0) {
11        int batchSize = msg->par("batchSize");
12        int userClass = msg->par("userClass");
13        if (userClass == 1)
14            numSystemClass1 -= batchSize;
15        else
16            numSystemClass2 -= batchSize;
17        vecSystemClass1.record(numSystemClass1);
18        vecSystemClass2.record(numSystemClass2);
19        delete msg;
20    }
21    else if (strcmp(msg->getName(), "ServerVacationEnded") ==
22              0) {
23        delete msg;
24        // controlla se soglia N raggiunta
25        if ((int)queueClass1.size() >= N) {
26            startSetUp();
27        }
28        else{
29            EV << "Attesa_raggiungimento_soglia_N";
30            waitingForUsers = true;
31        }
32    }
33    else if(strcmp(msg->getName(), "SourceBatchComplete")==0)
34    {
35        delete msg;
36        batchInProgress = false;
37        vecSystemClass1.record(numSystemClass1);
38        vecSystemClass2.record(numSystemClass2);
39        // se stiamo aspettando nuovi utenti, check soglia N
40        if (waitingForUsers && (int)queueClass1.size() >= N)
41        {
42            waitingForUsers = false;
43        }
44    }
45 }
```

```

40         startSetUp();
41     }
42     else if(waitingForBatchComplete){
43         waitingForBatchComplete = false;
44         serveBatch();
45     }
46 }
47 else {
48     // messaggio dalla Source
49     Job *job = check_and_cast<Job*>(msg);
50     batchInProgress = true;
51
52     if (job->getUserClass() == 1){
53         queueClass1.push_back(job);
54         numSystemClass1++;
55     }
56     else{
57         queueClass2.push_back(job);
58         numSystemClass2++;
59     }
60 }
61 }

```

Il modulo aggiorna costantemente le variabili che rappresentano il **numero di utenti nel sistema** per ciascuna classe, registrandone l'andamento, così da consentire l'analisi delle prestazioni del sistema.

La funzione `serveBatch()` implementa la service rule. Il tempo di servizio del batch viene calcolato direttamente nella coda (per semplicità implementativa, dato l'accesso immediato al contenuto delle code). Tale valore viene inserito come parametro nei job inviati al server, che si limita a rispettare il tempo ricevuto.

Listing 3: Funzione `serveBatch()` del modulo Queue.

```

1 void Queue::serveBatch()
2 {
3     int n1 = queueClass1.size();
4     int n2 = queueClass2.size();
5     std::vector<Job*> batch;
6     int r = 0;
7     double serviceTime = 0;
8

```

```

9      if (n1 >= a1) {
10          r = std::min(n1, b1);
11          for (int i = 0; i < r; ++i) {
12              batch.push_back(queueClass1.front());
13              queueClass1.pop_front();
14          }
15
16          // serviceTime classe 1
17          double s = (r / double(b1)) * sp1;
18          double q = uniform(l1, u1);
19          serviceTime = s + q;
20      }
21      else if (n2 >= a2) {
22          r = std::min(n2, b2);
23          for (int i = 0; i < r; ++i) {
24              batch.push_back(queueClass2.front());
25              queueClass2.pop_front();
26          }
27
28          // serviceTime classe 2
29          double s = (r / double(b2)) * sp2;
30          double q = uniform(l2, u2);
31          serviceTime = s + q;
32      }
33      else {
34          startVacation();
35          return;
36      }
37
38      // Invio del batch al server
39      for (auto job : batch) {
40          job->addPar("serviceTime") = serviceTime; // tutti i
41              job del batch usano lo stesso serviceTime
42          send(job, "out");
43      }
44      cMessage *batchDone = new cMessage("BatchComplete");
45      send(batchDone, "out");

```

Qualora non siano soddisfatte le condizioni minime per il servizio, la coda notifica al Server di iniziare una fase di vacation, comunicandogli anche la relativa durata.

Listing 4: Funzione startVacation() del modulo Queue.

```
1 void Queue::startVacation()
2 {
3     int k_class1 = queueClass1.size();
4     double t1 = ((k_class1 + 1) / double(b1)) * sv;
5     double t2 = uniform(z, w);
6     double vacationTime = t1 + t2;
7     cMessage *vacationMsg = new cMessage("StartVacation");
8     vacationMsg->addPar("vacationTime") = vacationTime;
9     send(vacationMsg, "out");
10 }
```

Al termine della vacation, il controllo sulla soglia N viene effettuato sempre nella coda. Quando il numero di utenti di classe 1 raggiunge la soglia, viene avviata la fase di setup (startSetup()). Anche in questo caso, la durata viene notificata al server tramite messaggio, mantenendo separata la logica decisionale (Queue) dall'esecuzione temporale effettiva (Server).

Listing 5: Funzione startSetup() del modulo Queue.

```
1 void Queue::startSetup()
2 {
3     double setupTime = exponential(h);
4     cMessage *setupMsg = new cMessage("StartSetup");
5     setupMsg->addPar("setupTime") = setupTime;
6     send(setupMsg, "out");
7 }
```

3.4 Componente Server

Il componente **Server** è responsabile dell'esecuzione temporale delle attività di servizio, vacation e setup, mentre la logica decisionale rimane delegata alla **Queue**. Il comportamento è implementato nel metodo **handleMessage()**.

Quando il server riceve i job di un batch, questi vengono temporaneamente memorizzati nel vettore **batchJobs**. La ricezione del messaggio **BatchComplete** indica che tutti i job del lotto sono arrivati; a questo punto viene rispettato il tempo di servizio (precedentemente calcolato dalla Queue) tramite il metodo **batchService()**. Tutti i job appartenenti allo stesso batch avranno quindi gli stessi tempi di "inizio" e "fine" servizio, come se venissero serviti in parallelo.

Terminato il servizio di un batch, il server calcola la durata dell'intervallo di occupazione (utile per stimare il **fattore di utilizzo del sistema**), notifica alla coda l'uscita del batch tramite un messaggio `JobDeparture` (necessario per poter conteggiare correttamente il **numero di utenti presenti nel sistema**), e inoltra tutti i job al `Sink`. Successivamente, richiede un nuovo batch alla coda mediante il messaggio `ServerRequestBatch`.

Il server gestisce inoltre i tempi di *vacation* e *setup*. In entrambi i casi riceve dalla `Queue` un messaggio contenente la durata della fase; il server si limita a schedulare l'evento di termine (`VacationEnd` o `SetupEnd`) e, alla sua occorrenza, notifica la coda. In particolare, al termine del setup viene immediatamente richiesto un nuovo batch da servire.

Questa **suddivisione delle responsabilità** consente di mantenere separata la logica di controllo (implementata nella `Queue`) dalla gestione temporale degli eventi (implementata nel `Server`).

Listing 6: Funzione `handleMessage()` del modulo `Server`.

```
1 void Server::handleMessage(cMessage *msg)
2 {
3     if (msg == endService) {
4         //Per stima fattore di utilizzo
5         simtime_t busyDuration = simTime() - lastBusyStart;
6         utilizationVector.record(busyDuration);
7
8         //Notifica alla coda che un job sta uscendo dal
9         //sistema (per statistiche)
10        int batchSize = batchJobs.size();
11        int uClass = batchJobs.front()->getUserClass();
12        cMessage *batchDeparture = new cMessage("JobDeparture");
13        batchDeparture->addPar("batchSize") = batchSize;
14        batchDeparture->addPar("userClass") = uClass;
15        send(batchDeparture, "outQueue");
16
17        // Invia tutti i job del batch al sink
18        for (auto job : batchJobs) {
19            send(job, "outSink");
20        }
21        batchJobs.clear();
22
23        // Chiedi un nuovo batch alla coda
24        send(new cMessage("ServerRequestBatch"), "outQueue");
```

```

24     }
25     else if (strcmp(msg->getName(), "StartVacation") == 0){
26         vacationStart = simTime();
27         simtime_t vacationTime = msg->par("vacationTime").
            doubleValue();
28         delete msg;
29         cMessage *vacationEnd = new cMessage("VacationEnd");
30         scheduleAt(simTime() + vacationTime, vacationEnd);
31     }
32     else if (strcmp(msg->getName(), "VacationEnd") == 0) {
33         //Calcolo durata vacation (per statistiche)
34         simtime_t vacationDuration = simTime() -
            vacationStart;
35         vecVacationDurations.record(vacationDuration);
36
37         //Notifica alla coda
38         send(new cMessage("ServerVacationEnded"), "outQueue")
            ;
39         delete msg;
40     }
41     else if (strcmp(msg->getName(), "StartSetup") == 0) {
42         simtime_t setupTime = msg->par("setupTime").
            doubleValue();
43         delete msg;
44         cMessage *setupEnd = new cMessage("SetupEnd");
45         scheduleAt(simTime() + setupTime, setupEnd);
46     }
47     else if (strcmp(msg->getName(), "SetupEnd") == 0) {
48         send(new cMessage("ServerRequestBatch"), "outQueue");
49         delete msg;
50     }
51     else if (strcmp(msg->getName(), "BatchComplete") == 0) {
52         delete msg;
53         batchService();
54     }
55     else{
56         Job *job = check_and_cast<Job*>(msg);
57         batchJobs.push_back(job);
58     }
59 }

```

Listing 7: Funzione `batchService()` del modulo `Server`.

```
1 void Server::batchService()  
2 {  
3     lastBusyStart = simTime();  
4  
5     // Recupero il serviceTime da uno dei job  
6     simtime_t serviceTime = batchJobs.front()->par("  
7         serviceTime").doubleValue();  
8  
9     // Schedule fine servizio per tutto il batch  
10    scheduleAt(simTime() + serviceTime, endService);  
}
```

3.5 Interazione tra Queue e Server

Nelle Figure 2 e 3 è mostrata, a scopo chiarificatore, l'interazione tra i componenti `Queue` e `Server`, evidenziando i messaggi scambiati durante il funzionamento del sistema.

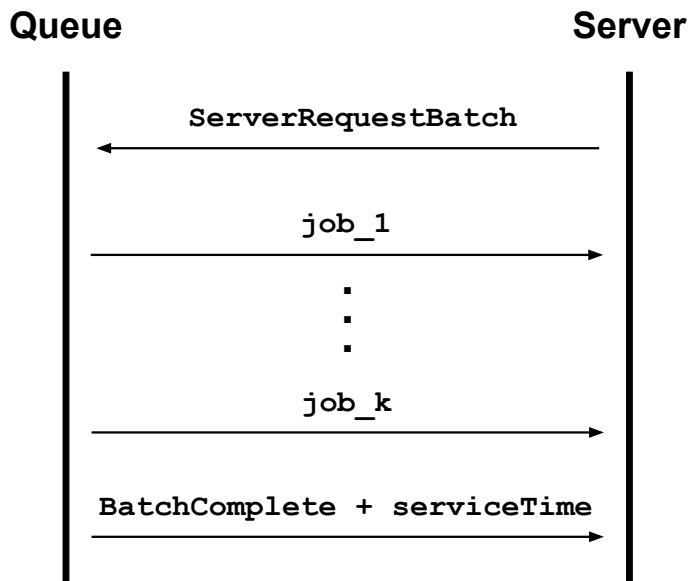


Figura 2: Scambio di messaggi tra `Queue` e `Server` nel momento in cui il server richiede un nuovo batch da servire.

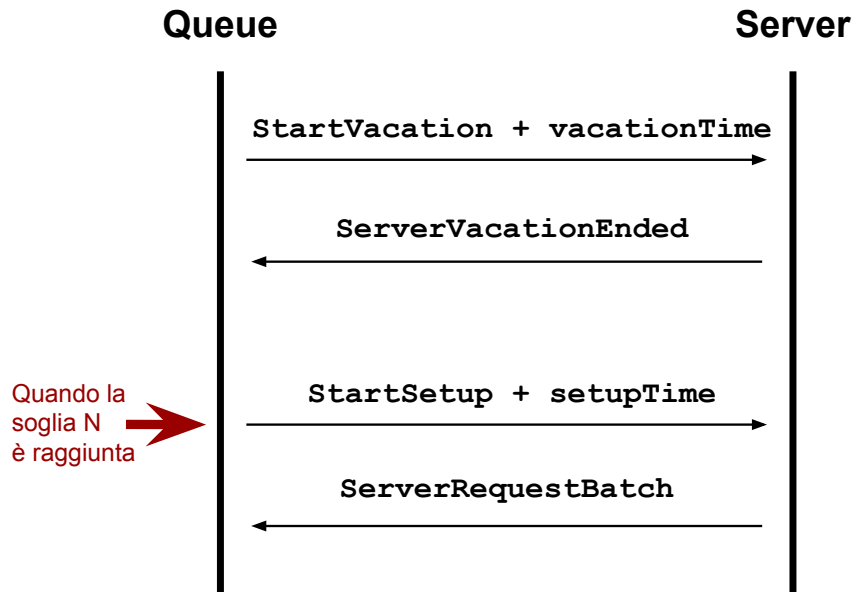


Figura 3: Scambio di messaggi tra Queue e Server durante le fasi di vacation e set-up.

3.6 Componente Sink

Il componente **Sink** rappresenta il punto di uscita del sistema. Il suo comportamento è estremamente semplice: per ogni job ricevuto dal Server, il **Sink** registra il **tempo di permanenza nel sistema**, calcolato come differenza tra l'istante corrente di simulazione (`simTime()`) e il tempo di arrivo precedentemente salvato nel campo `arrivalTime`.

Dopodichè, il job viene eliminato, simulando la sua uscita definitiva dal sistema.

Listing 8: Funzione `handleMessage()` del modulo Sink.

```

1 void Sink::handleMessage(cMessage *msg)
2 {
3     Job *job = check_and_cast<Job*>(msg);
4     simtime_t timeInSystem = simTime() - job->getArrivalTime
5     ();
6     if (job->getUserClass() == 1)
7         vecTimeClass1.record(timeInSystem);
8     else
9         vecTimeClass2.record(timeInSystem);
10    delete job;
11 }

```

4 Stima delle misure di prestazione

In questa sezione viene descritta la metodologia utilizzata per determinare la fine del transiente iniziale (*warm-up*) e per calcolare le stime puntuali e gli intervalli di confidenza delle seguenti misure di prestazione del sistema:

- il numero medio di utenti di classe i ($i = 1, 2$) presenti nel sistema;
- il tempo medio di permanenza nel sistema di un utente di classe i ($i = 1, 2$);
- il fattore di utilizzo del sistema;
- la durata media dei periodi di *vacation*.

La metodologia adottata si basa sulle tecniche descritte in [3].

Nel contesto di questo progetto, i calcoli sono stati implementati in Python utilizzando le librerie NumPy e Pandas. Il codice sorgente è disponibile nel seguente [Notebook](#).

4.1 Dati raccolti durante le simulazioni

Per ciascuna **configurazione del parametro m** (come descritto nella Sezione 2.8) sono state effettuate **20 run indipendenti**, ciascuna della **durata di 15000s**.

I dati prodotti sono stati esportati in file CSV, organizzati per ciascuna metrica sotto forma di coppie (`timestamp`, `value`):

- `NumUsersSystemClass1` e `NumUsersSystemClass2`: `value` indica il numero assoluto di utenti rispettivamente di classe 1 e classe 2 presenti nel sistema all'istante `timestamp`.
- `TimeInSystemClass1` e `TimeInSystemClass2`: `value` rappresenta il tempo di permanenza nel sistema dell'utente che ha completato il servizio all'istante `timestamp`.
- `Utilization`: `value` corrisponde alla durata di un periodo **busy** terminato all'istante `timestamp` (notare che tale valore non rappresenta ancora il fattore di utilizzo del sistema, ma solo la lunghezza degli intervalli di occupazione del server).
- `VacationDurations`: `value` indica la durata di un periodo di *vacation* concluso all'istante `timestamp`.

4.2 Analisi del transiente iniziale

4.2.1 Il problema del transiente iniziale

Nelle simulazioni di sistemi a eventi discreti, e in particolare nelle reti di code, le **condizioni iniziali** influenzano in modo significativo l'andamento delle osservazioni nelle fasi iniziali della simulazione. Nel caso delle simulazioni svolte per il presente progetto, il sistema è inizialmente vuoto; questo genera un periodo iniziale in cui le grandezze di interesse risultano distorte rispetto al **regime stazionario**.

Durante tale **fase transiente**, le stime delle prestazioni (tempi di permanenza nel sistema, numero medio di utenti nel sistema, ecc.) risultano affette da bias. Per evitare tale problema, è necessario individuare un opportuno **periodo di warm-up** da eliminare, in modo da effettuare le analisi statistiche esclusivamente sui dati raccolti in regime stazionario.

4.2.2 Individuazione del punto di warm-up

L'individuazione del periodo di warm-up è stata effettuata mediante il **metodo grafico di Welch**.

Come accennato in precedenza, per ciascuna configurazione del parametro m sono state eseguite 20 run indipendenti di simulazione. Per ogni metrica di interesse è stata costruita, per ogni istante t , la **media tra repliche**:

$$\hat{\mu}(t) = \frac{1}{20} \sum_{r=1}^{20} X_r(t),$$

dove $X_r(t)$ rappresenta il valore osservato alla replica r all'istante t .

L'estimatore $\hat{\mu}(t)$ è una variabile aleatoria; assumendo repliche indipendenti e identicamente distribuite, risulta

$$E[\hat{\mu}(t)] = \mu(t),$$

ossia esso fornisce una stima non distorta del valore medio del processo al tempo t .

Se il sistema è stabile, la media del processo converge, al crescere del tempo, al valore atteso in regime stazionario μ_{staz} . Di conseguenza, per istanti sufficientemente grandi, anche $\hat{\mu}(t)$ tende a stabilizzarsi attorno a μ_{staz} . L'analisi grafica dell'andamento di $\hat{\mu}(t)$ consente quindi di individuare il termine del transiente iniziale.

Nelle simulazioni a eventi discreti, gli istanti di osservazione dipendono dalla dinamica degli eventi e possono quindi differire tra una replica e l'altra. Di conseguenza, le traiettorie temporali delle diverse run non risultano naturalmente allineate su una stessa griglia temporale (ovvero, per ogni run potrebbero essere stati raccolti dati in

timestamp diversi). Dunque, per consentire il calcolo della media punto per punto tra repliche, ciascuna traiettoria è stata **interpolata su una griglia temporale comune**. Tale procedura ha permesso di associare, a ogni istante della griglia, un valore per ciascuna replica e di calcolare in modo coerente la media $\hat{\mu}(t)$.

Per attenuare le fluttuazioni a breve termine e rendere più evidente la convergenza al regime stazionario, è stato applicato uno **smoothing** mediante una media mobile centrata.

Indicando con $\hat{\mu}(t)$ la media tra repliche, la curva smussata $\tilde{\mu}(t; K)$ è stata calcolata come

$$\tilde{\mu}(t; K) = \frac{1}{2K+1} \sum_{i=t-K}^{t+K} \hat{\mu}(i), \quad \text{per } t \geq K$$

La semi-finestra è stata scelta pari a

$$K = 1.6\sqrt{T}$$

dove T rappresenta la durata complessiva della simulazione.

Per gli istanti iniziali, per i quali non è possibile disporre di K osservazioni precedenti, la media mobile è stata calcolata su una finestra progressivamente crescente:

$$\tilde{\mu}(t; K) = \frac{1}{2t+1} \sum_{i=0}^{2t} \hat{\mu}(i), \quad \text{per } t < K$$

In questo modo si è ottenuta una curva smussata $\tilde{\mu}(t; K)$, ovvero una curva più regolare che facilita l'individuazione del periodo di warm-up.

Tradizionalmente, il metodo di Welch è un approccio grafico per individuare il periodo di warm-up osservando l'andamento della media tra repliche. In questo lavoro, tale metodo è stato **automatizzato** mediante un criterio numerico.

In particolare, la media stazionaria di riferimento è stata stimata considerando l'ultimo 20% della curva smussata. Sulla base di questa stima, è stata definita una **banda di tolleranza** pari all'1.5% del valore medio finale. Il punto di warm-up per ciascuna metrica è stato identificato come il primo istante a partire dal quale la curva smussata rimane stabilmente all'interno di tale banda per un numero consecutivo di punti pari a una **finestra** prefissata.

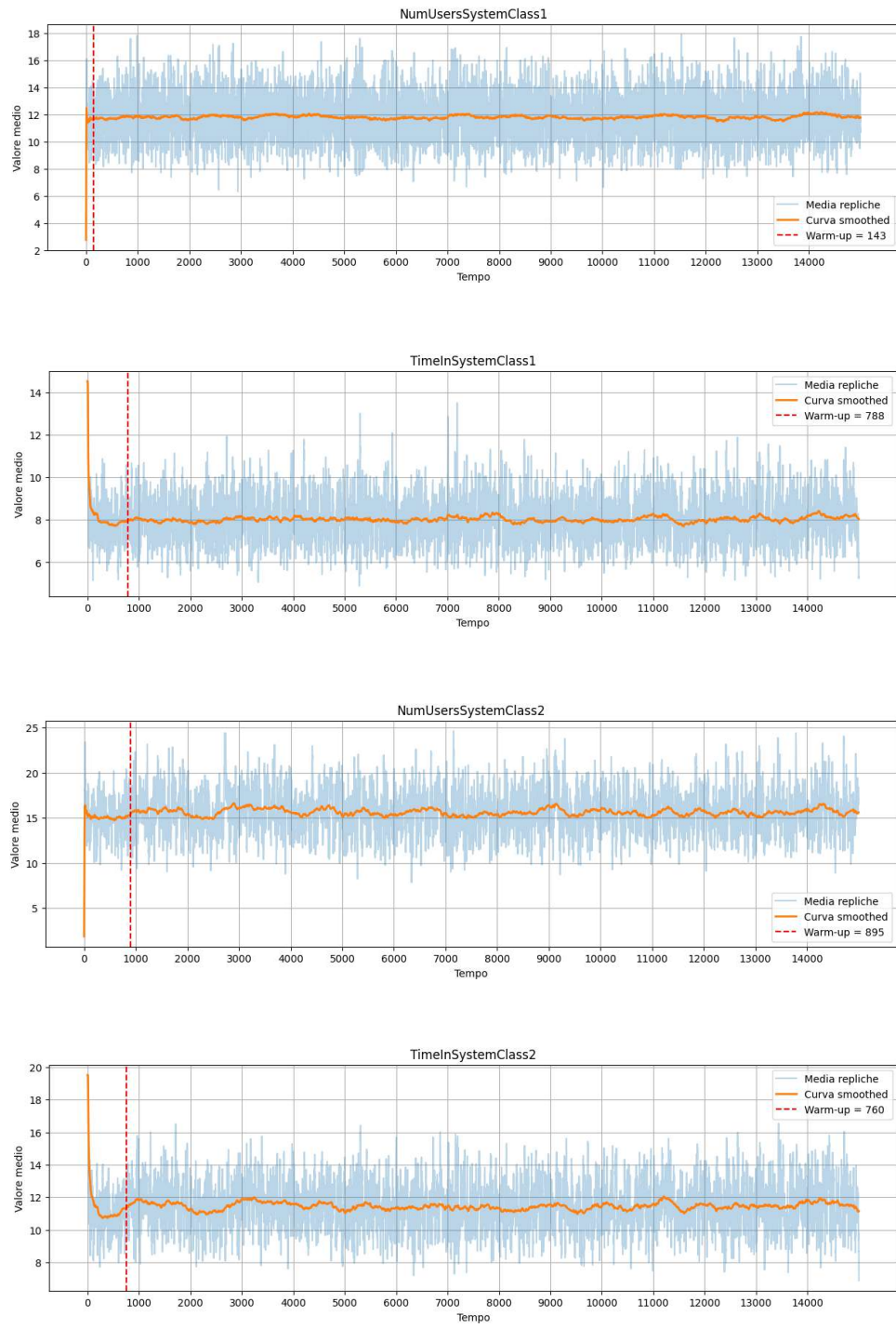


Figura 4: Individuazione del periodo di warm-up sui grafici del numero di utenti e del tempo di permanenza nel sistema. Esempio con $m = 1.4$

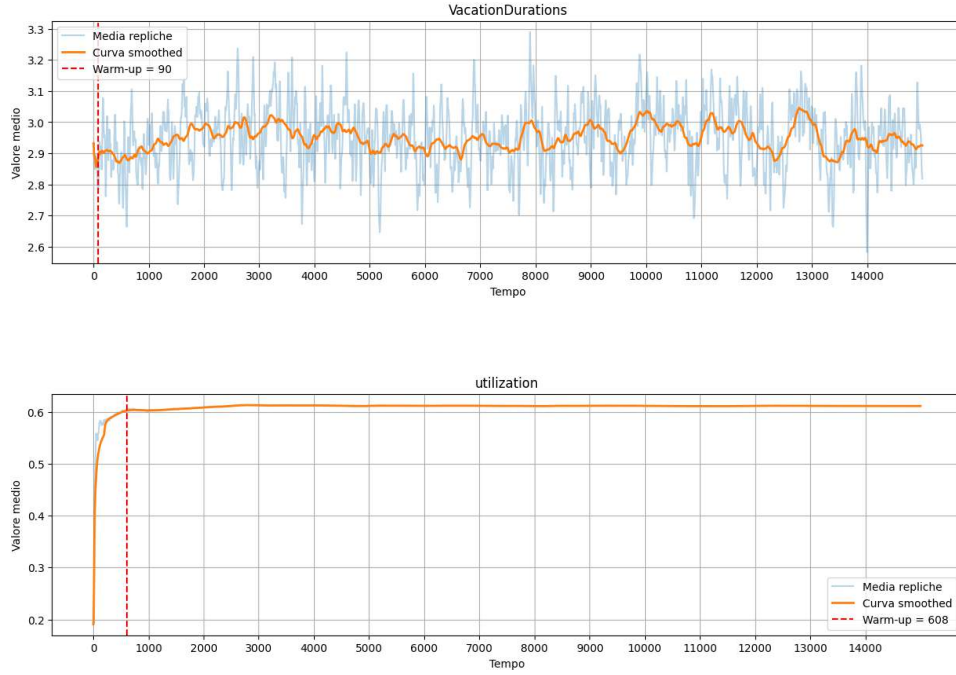


Figura 5: Individuazione del periodo di warm-up sui grafici della durata dei periodi di vacation e del fattore di utilizzo (media cumulativa) del sistema. Esempio con $m = 1.4$

Poiché metriche diverse possono raggiungere il regime stazionario in istanti differenti, è stato assunto come tempo di fine transiente globale il **massimo tra i warm-up stimati** per ciascuna metrica:

$$t_w = \max_i t_w^{(i)}.$$

Per garantire una maggiore sicurezza nella rimozione del transiente iniziale, è stato infine introdotto un **margin** pari al **20%** del valore stimato, ottenendo il tempo finale di warm-up utilizzato nell'analisi:

$$t_w^{\text{final}} = 1.2 t_w.$$

Infine, tutti i dati raccolti per $t < t_w^{\text{final}}$ sono stati rimossi, in modo che il calcolo delle stime puntuali e degli intervalli di confidenza fosse basato esclusivamente su osservazioni in regime stazionario. Questa procedura ha permesso di ottenere statistiche più affidabili e rappresentative del comportamento a regime del sistema.

4.3 Calcolo delle stime puntuali

Le misure di prestazioni del sistema sono state stimate mediante il metodo delle **repliche indipendenti**.

Per ciascuna metrica, dopo la rimozione dei dati relativi al periodo di warm-up ($t < t_w^{\text{final}}$), è stata calcolata una stima per ciascuna run $r = 1, \dots, R$ (con $R = 20$), di seguito indicata con Y_r .

La modalità di calcolo di Y_r dipende dalla natura della metrica considerata:

- **Fattore di utilizzo del sistema.** Il fattore di utilizzo è stato stimato come rapporto tra il tempo totale di occupazione del sistema e la durata del periodo di osservazione:

$$Y_r = \frac{\text{busy_time}_r}{T - t_w^{\text{final}}}.$$

- **Numero medio di utenti nel sistema.** Nella simulazione, il numero di utenti viene registrato solo negli istanti in cui si verifica una variazione (arrivo di un batch o completamento di un servizio); tra due timestamp consecutivi il valore della metrica rimane costante. Di conseguenza, le osservazioni non sono equispaziate nel tempo e una semplice media aritmetica dei valori registrati non sarebbe rappresentativa del comportamento del sistema, poiché attribuirebbe lo stesso peso a intervalli temporali di diversa durata.

Per ottenere una stima corretta della prestazione è stata quindi calcolata la media temporale (*time-average*), definita come

$$Y_r = \frac{1}{T - t_w^{\text{final}}} \int_{t_w^{\text{final}}}^T X_r(t) dt.$$

Operativamente, tale integrale è stato approssimato tramite una somma pesata sui sottointervalli tra eventi consecutivi:

$$Y_r \approx \frac{\sum_i X_r(t_i) (t_{i+1} - t_i)}{T - t_w^{\text{final}}},$$

dove t_i rappresentano gli istanti in cui il numero di utenti cambia valore.

In questo modo, ciascun valore osservato contribuisce alla media in proporzione alla durata dell'intervallo temporale in cui rimane valido, fornendo una stima corretta della grandezza in regime stazionario.

- **Tempo di permanenza nel sistema e durata del periodo di vacation.** Per le metriche rimanenti è stata semplicemente calcolata la media

campionaria dei valori osservati a seguito della rimozione del transiente:

$$Y_r = \frac{1}{n_r} \sum_{i=1}^{n_r} X_{r,i}.$$

Dove n_r rappresenta il numero di osservazioni disponibili nella replica r dopo l'eliminazione della fase transiente.

Una volta ottenute le R stime indipendenti $\{Y_1, \dots, Y_R\}$, la stima puntuale della prestazione è stata calcolata come media tra repliche:

$$\bar{Y} = \frac{1}{R} \sum_{r=1}^R Y_r.$$

Poiché le repliche sono indipendenti e identicamente distribuite, $\bar{Y}$ costituisce uno stimatore non distorto della media stazionaria della metrica considerata.

La variabilità della stima è stata valutata calcolando la varianza campionaria tra repliche:

$$S^2 = \frac{1}{R-1} \sum_{r=1}^R (Y_r - \bar{Y})^2,$$

da cui si ottiene la deviazione standard campionaria:

$$S = \sqrt{S^2}.$$

La scelta del denominatore $R-1$ garantisce che la varianza stimata sia uno stimatore non distorto della varianza reale tra repliche.

4.4 Calcolo degli intervalli di confidenza

Un intervallo di confidenza è un intervallo casuale che, con probabilità $1-\alpha$, contiene il vero valore del parametro di interesse. Nel contesto del metodo delle repliche indipendenti, si considerano le R stime indipendenti $\{Y_1, \dots, Y_R\}$ ottenute dopo la rimozione del periodo di warm-up.

Poiché la varianza reale della metrica è ignota e disponiamo unicamente della varianza campionaria precedentemente calcolata, la distribuzione della statistica standardizzata (che misura l'errore tra la media stimata e il valore reale):

$$\frac{\bar{Y} - \mu}{S/\sqrt{R}}$$

non segue esattamente una distribuzione normale standard, bensì una distribuzione t di Student.

La differenza nasce dal fatto che la deviazione standard reale σ non è nota e viene stimata tramite S . Questa sostituzione introduce un'ulteriore fonte di incertezza: oltre alla variabilità dovuta al campionamento, occorre tenere conto anche dell'errore associato alla stima della variabilità stessa. La distribuzione t di Student tiene conto di tale incertezza aggiuntiva, presentando code più larghe rispetto alla normale standard e producendo quindi intervalli di confidenza leggermente più ampi.

L'intervallo di confidenza al livello $1 - \alpha$ è pertanto dato da

$$\bar{Y} \pm t_{1-\alpha/2, R-1} \frac{S}{\sqrt{R}},$$

dove $t_{1-\alpha/2, R-1}$ rappresenta il quantile della distribuzione t di Student calcolato sulla base del numero di repliche disponibili.

All'aumentare di R , la stima di σ diventa più accurata e la distribuzione t tende progressivamente alla distribuzione normale standard, riducendo la differenza tra i due approcci.

Nel presente studio è stato scelto $\alpha = 0.05$, ottenendo intervalli di confidenza al 95%.

5 Risultati

In questa sezione vengono presentati i principali risultati ottenuti dalle simulazioni.

5.1 Transiente iniziale

Nelle Tabelle 2 e 3 sono riportati i valori del *warm-up* calcolati secondo l'algoritmo descritto nella sezione precedente. Le stime puntuali e gli intervalli di confidenza delle metriche di prestazione sono stati successivamente calcolati escludendo i dati corrispondenti al periodo iniziale pari a "Warm-up + 20%", indicato nell'ultima riga della Tabella 3.

Tabella 2: Tempi di warm-up (secondi) calcolati per ciascuna metrica al variare di m

Metrica	$m = 1.0$	$m = 1.4$	$m = 2.0$	$m = 2.5$
NumUsersSystemClass1	57	143	224	212
NumUsersSystemClass2	197	895	115	817
VacationDurations	403	90	74	0
Utilization	1420	608	652	839
TimeInSystemClass1	203	788	218	204
TimeInSystemClass2	456	760	239	385

Tabella 3: Tempi di warm-up (secondi) finali, con e senza margine aggiuntivo del 20%, al variare di m

	$m = 1.0$	$m = 1.4$	$m = 2.0$	$m = 2.5$
Warm up	1420	895	652	839
Warm up + 20%	1704	1074	782	1007

5.2 Misure di prestazione

Di seguito sono riportati i valori delle stime puntuali e dei corrispondenti intervalli di confidenza al 95% ($\alpha = 0.05$), calcolati per ciascuna configurazione del parametro m .

5.2.1 Stime per $m = 1.0$

Tabella 4: Stime puntuali, deviazione standard e intervalli di confidenza (95%) per ciascuna misura di prestazione, per $m = 1.0$

Metrica	Stima puntuale	Deviazione std.	Intervallo di confidenza
Numero utenti (classe 1)	12.4493	0.1241	[12.3912 , 12.5074]
Numero utenti (classe 2)	24.9162	1.1932	[24.3578 , 25.4746]
Durata vacation	2.9641	0.0268	[2.9515 , 2.9766]
Fattore di utilizzo	0.8549	0.0070	[0.8516 , 0.8581]
T. permanenza (classe 1)	4.9645	0.0412	[4.9453 , 4.9838]
T. permanenza (classe 2)	12.4309	0.5592	[12.1692 , 12.6926]

5.2.2 Stime per $m = 1.4$

Tabella 5: Stime puntuali, deviazione standard e intervalli di confidenza (95%) per ciascuna misura di prestazione, per $m = 1.4$

Metrica	Stima puntuale	Deviazione std.	Intervallo di confidenza
Numero utenti (classe 1)	11.8868	0.0988	[11.8406 , 11.9331]
Numero utenti (classe 2)	15.6964	0.3363	[15.5390 , 15.8538]
Durata vacation	2.9541	0.0185	[2.9455 , 2.9628]
Fattore di utilizzo	0.6122	0.0073	[0.6088 , 0.6156]
T. permanenza (classe 1)	6.6510	0.0649	[6.6207 , 6.6814]
T. permanenza (classe 2)	10.9988	0.2084	[10.9013 , 11.0963]

5.2.3 Stime per $m = 2.0$

Tabella 6: Stime puntuali, deviazione standard e intervalli di confidenza (95%) per ciascuna misura di prestazione, per $m = 2.0$

Metrica	Stima puntuale	Deviazione std.	Intervallo di confidenza
Numero utenti (classe 1)	11.4671	0.0790	[11.4301 , 11.5041]
Numero utenti (classe 2)	13.3050	0.2403	[13.1925 , 13.4174]
Durata vacation	2.9489	0.0255	[2.9370 , 2.9609]
Fattore di utilizzo	0.4210	0.0054	[0.4185 , 0.4236]
T. permanenza (classe 1)	9.1248	0.0881	[9.0836 , 9.1661]
T. permanenza (classe 2)	13.3224	0.1899	[13.2335 , 13.4112]

5.2.4 Stime per $m = 2.5$

Tabella 7: Stime puntuali, deviazione standard e intervalli di confidenza (95%) per ciascuna misura di prestazione, per $m = 2.5$

Metrica	Stima puntuale	Deviazione std.	Intervallo di confidenza
Numero utenti (classe 1)	11.2195	0.0755	[11.1842 , 11.2548]
Numero utenti (classe 2)	12.5134	0.2768	[12.3839 , 12.6430]
Durata vacation	2.9544	0.0267	[2.9419 , 2.9669]
Fattore di utilizzo	0.3318	0.0048	[0.3295 , 0.3341]
T. permanenza (classe 1)	11.2403	0.1750	[11.1584 , 11.3222]
T. permanenza (classe 2)	15.5588	0.2879	[15.4241 , 15.6936]

5.3 Osservazioni sui risultati e conclusioni

5.3.1 Osservazioni

Dall'analisi dei risultati delle simulazioni emergono alcune osservazioni interessanti.

Innanzitutto, all'aumentare di m , ossia con tempi medi di inter-arrivo più lunghi, il numero medio di utenti nel sistema diminuisce per entrambe le classi, coerentemente con quanto ci si aspetterebbe: tempi di arrivo più lunghi comportano un minore carico sul server, riducendo la congestione.

Parallelamente, il fattore di utilizzo ρ del server diminuisce significativamente (da 0.8549 per $m = 1.0$ a 0.3318 per $m = 2.5$), confermando che il server diventa progressivamente meno carico e che il sistema rimane stabile in tutti i casi considerati.

Va inoltre considerato che gli utenti di classe 1 vengono serviti con priorità maggiore e che il server è soggetto a periodi di *vacation*. Questo comportamento influisce sul tempo medio di permanenza degli utenti nel sistema, che per entrambe le classi di utenti tende ad aumentare all'aumentare di m . Tale fenomeno può essere spiegato intuitivamente:

- All'aumentare della media dei tempi di inter-arrivo m , gli utenti arrivano più lentamente.
- Di conseguenza, la coda contiene in media meno utenti.
- Poiché il server entra in *vacation* quando la coda ha pochi utenti, con inter-arrivi più lunghi è probabile che il server vada in *vacation* più frequentemente.
- Questo implica che gli utenti che sono invece presenti nel sistema possono dover attendere più a lungo prima di essere serviti.

La durata media delle *vacation*, invece, risulta simile per tutte le configurazioni.

5.3.2 Controllo della coerenza dei risultati

Una prova interessante consiste nell'osservare che **il sistema simulato è effettivamente stabile** e che **la legge di Little è rispettata** [4].

Ad esempio, consideriamo la configurazione $m = 1.0$: poiché nel sistema è presente un singolo server con fattore di utilizzo stimato $\rho = 0.8549$, e dato che $\rho < 1$, possiamo affermare che il server non è sovraccarico e che il sistema può essere considerato stabile.

In un sistema stabile è possibile applicare la legge di Little, che stabilisce che, a regime stazionario, per ciascuna classe i di utenti il numero medio di utenti nel

sistema L_i è uguale al prodotto del tasso medio di arrivo λ_i e del tempo medio di permanenza W_i :

$$L_i = \lambda_i W_i.$$

Per una distribuzione uniforme discreta $U[a, b]$, il valore atteso è:

$$E[B] = \frac{a + b}{2}.$$

Applicando questa formula otteniamo il numero medio di utenti per batch di ciascuna classe:

$$E[B_1] = \frac{1 + 4}{2} = 2.5,$$

$$E[B_2] = \frac{1 + 3}{2} = 2.$$

Di conseguenza, il tasso medio di arrivo degli utenti di ciascuna classe (ricordando che gli utenti arrivano in batch, quindi il tasso effettivo di arrivo dei singoli utenti differisce dal parametro m) è:

$$\lambda_1 = \lambda_{\text{batch}} \cdot E[B_1] = 1 \cdot 2.5 = 2.5,$$

$$\lambda_2 = \lambda_{\text{batch}} \cdot E[B_2] = 1 \cdot 2 = 2.$$

Per confermare la coerenza del modello e della simulazione, si confrontano i valori previsti dalla legge di Little con i risultati della simulazione. Dati i tempi medi di permanenza stimati:

$$W_1 = 4.9645, \quad W_2 = 12.4309,$$

i valori previsti del numero medio di utenti nel sistema sono:

$$L_1^{\text{pred}} = \lambda_1 \cdot W_1 = 2.5 \cdot 4.9645 \approx 12.411,$$

$$L_2^{\text{pred}} = \lambda_2 \cdot W_2 = 2 \cdot 12.4309 \approx 24.862.$$

Questi valori risultano molto vicini alle stime puntuali della simulazione:

$$L_1 = 12.4493, \quad L_2 = 24.9162,$$

Pertanto, sia la stabilità del sistema sia la validità della legge di Little sono confermate dai risultati della simulazione.

Riferimenti bibliografici

- [1] P. Karan and S. Pradhan. Analysis of a queue-length-dependent vacation queue with bulk service, n-policy, set-up time and cost optimization. *Perform. Evaluation*, 167:102459, 2025.
- [2] OpenSim Ltd. OMNeT++ 6.0 Simulation Manual. <https://doc.omnetpp.org/omnetpp/>, 2023.
- [3] Steven S. Lavenberg. Chapter 6 - the statistical analysis of simulation results. In *Computer Performance Modeling Handbook*, pages 281–312. Academic Press, 1983.
- [4] Christos G. Cassandras and Stéphane Lafortune. *Introduction to Discrete Event Systems*. Springer, 2 edition, 2008.